ROCKETRIDE EXPLAINED / 01

Answers before the meeting

RocketRide Redis integration | A technical companion to your meeting guide | October 29, 2026

Several earlier meeting questions have answers in RocketRide's code. This document explains those answers at a basic Python level, with actual request and result examples. Use the meeting to settle requirements and ownership rather than to rediscover the framework.

Question	What we can already answer
Is there an example node?	Yes: tool_notion is a Python tool node. Its implementation, manifest, and tests are available. Pages 2-4.
How does it receive and return data?	Named Python methods receive an argument dictionary and return structured dictionaries. Page 3.
How can I run it?	Offline tests run without an engine or Notion account. A live call requires an engine and Notion credentials. Pages 4-5.
Must an AI agent call the node?	No. A Tools-source pipeline lets application code call tools directly through client.tool(). Page 5.
What library should Redis use?	The brief and source lock agree on redis-py 7.4.1. Existing nodes already use it. Page 6.
What testing and app workflow exist?	RocketRide documents contract/live tests, App Builder, and the catalog contribution process. Pages 4 and 7.

What was verified: 78 Notion unit tests and 2 manifest contract checks passed. The real Notion method was also exercised with a mocked external API. No live Notion workspace, Redis node, or full RocketRide runtime was exercised in this preparation.

Reading order: pages 2-3 explain the node, page 4 shows tests you can run, page 5 shows the documented live-call route, and pages 6-8 connect those answers to Redis and the remaining meeting decisions.

References point to source commit 40ea9ecbbfabbed7d4d7250415ef8b769be5a0bc. A fresh fetch found one newer develop commit affecting only the docs footer, so the node and SDK documentation used here are unchanged. [A]

ROCKETRIDE EXPLAINED / 02

What does an example node contain?

Answer: a small Python package plus a manifest describing how RocketRide loads and presents it.

The example is **nodes/src/nodes/tool_notion/** in the actual rocketride-server repository. It exposes eight operations, including search, reading pages, and creating or updating content. We will follow one read operation, notion_search. [A]

File	Its role
services.json	The manifest: name, Python module path, tool capability, connection settings, and editor fields.
__init__.py	Exports IGlobal and IInstance so the engine can load them.
IGlobal.py	Reads shared settings and validates the credential. This is lifecycle setup, not the function for each request.
IInstance.py	Defines callable operations. notion_search is a method here.
notion_client.py	Makes the external HTTP requests and handles provider errors and retries.
requirements.txt	Declares the provider dependency. Notion uses requests; our Redis node would use redis-py.
README.md	Explains configuration, operations, and limitations for users.

What makes a Python method a tool?

The **@tool_function** decorator attaches an input schema, output schema, and description to a method. A schema describes fields and their types. The engine discovers these methods and dispatches tool calls to them. You still write validation and error handling inside the implementation.

Why does this node have no normal input lanes?

Its manifest declares classType=["tool"], the invoke capability, and empty lanes. It receives calls through a **control connection**, which attaches a callable capability, rather than streaming questions through a normal data lane. [A, B]

This node is Python even though the engine core is C++. Run it through the engine, not IInstance.py directly. Notion is marked experimental; its README says live end-to-end workspace verification is still missing. [A]

ROCKETRIDE EXPLAINED / 03

What goes in and comes out?

Answer: operation arguments go in as a dictionary; a result dictionary or a structured failure comes back.

Observed offline trace: the actual notion_search method was called with the arguments below. The external API was mocked to return one example page; that page is test data, not a live Notion record.

```
inst.notion_search({
    "query": "roadmap",
    "filter_type": "page",
    "page_size": 5
})
```

The method normalizes the arguments and builds this provider request. The client adds the credential header separately. [A]

```
POST https://api.notion.com/v1/search

{
    "query": "roadmap",
    "filter": {"property": "object", "value": "page"},
    "page_size": 5
}
```

The node converted the mocked response into this exact result:

```
{
    "success": true,
    "results": [{"id": "example-page"}],
    "has_more": false,
    "next_cursor": null
}
```

has_more tells the caller whether more results exist; **next_cursor** is a continuation token. Here, false and null mean no further page. In Python these are False and None; the block above is JSON.

What happens on bad input or provider failure?

Calling notion_get_database({}) returned success=false with an error saying database_id is required, without making an external request. A mocked HTTP 401 produced success=false and an unauthorized error. Missing configuration can instead fail node startup, and SDK/transport failures can raise exceptions. These are different failure stages.

ROCKETRIDE EXPLAINED / 04

How can I run the existing tests?

Answer: the Notion behavior tests can run in ordinary Python with mocked engine and API boundaries.

From the **rocketride-server repository root**, use an isolated environment. The following is the lightweight reference-test route used here, not a full engine installation. [A, D]

```
python3 -m venv .venv-reference
source .venv-reference/bin/activate
python -m pip install pytest pytest-asyncio \
    pytest-timeout requests

python -m pytest nodes/test/tool_notion/test_notion.py \
    --confcutdir=nodes/test/tool_notion -o addopts='' -q

python -m pytest nodes/test/test_contracts.py \
    --noconftest -k tool_notion -o addopts='' -q
```

The first command ran **78 passing tests**. The second ran **2 passing contract checks**, with 392 other cases deselected. A pytest fixture-loop deprecation warning was informational; it did not fail the run.

What the test options are doing

--confcutdir and **--noconftest** keep this focused run from loading the full repository test harness and its engine dependencies. **-k tool_notion** selects the relevant contract cases. **-o addopts=''** clears repository default command options. The Notion tests supply scoped stand-ins for engine-only imports.

Check	What a pass establishes
Unit tests	Argument handling, request construction, results, and errors behave as expected against controlled fake responses.
Contract checks	The selected node manifest and module contract meet the checks implemented by the repository.
Live integration test	A real engine can call a real service successfully. This has not been run here.

For the normal prepared contributor environment, the official commands include ./builder nodes:test-contracts and ./builder nodes:test-full. A passing mock test does not replace a real Redis run, authentication check, or TLS check.

ROCKETRIDE EXPLAINED / 05

How would I call the node live?

Answer: use a Tools source and client.tool(). A model or agent is not required for this route.

Prerequisites: a running engine with the tools and tool_notion providers, the RocketRide Python SDK, a workspace .env pointing to that engine, and a valid NOTION_API_KEY in the **runtime process that loads the node**. The Notion integration must have access to the pages it searches. [A, C, E]

This is a recipe assembled from the documented SDK and source, **not a live-verified run**. Empty node config below relies on that engine environment variable. A key placed only in the client terminal is not sufficient.

```
import asyncio
from rocketride import RocketRideClient

pipeline = {"source": "tools_1", "components": [
    {"id": "tools_1", "provider": "tools",
     "config": {}, "input": []},
    {"id": "notion_1", "provider": "tool_notion",
     "config": {}, "input": [],
     "control": [{"classType": "tool", "from": "tools_1"}]}
]}

async def main():
    async with RocketRideClient() as client:
        await client.validate(pipeline)
        run = await client.use(pipeline=pipeline)
        try:
            result = await client.tool(
                token=run["token"], tool="notion_search",
                node_id="notion_1", input={"query": "roadmap"})
            print(result)
        finally:
            await client.terminate(run["token"])

asyncio.run(main())
```

Read the call: use() starts the pipeline and returns a run token; node_id selects its Notion component; tool selects the method; input carries the argument dictionary. terminate() releases the run. The SDK returns the tool's result. [C, E]

Run from a proper SDK workspace, not your earlier issue-connector folder: that folder's own rocketride Python package can shadow the SDK. Confirm .env points at the intended server; the SDK otherwise has a cloud default.

ROCKETRIDE EXPLAINED / 06

What carries over to our Redis node?

Answer: reuse RocketRide's tool interface and lifecycle pattern, and replace Notion HTTP calls with Redis client calls.

We already know the implementation route

Use a Python package with a manifest, IGlobal/IInstance classes, tool methods, dependencies, and docs. The brief specifies **redis-py 7.4.1**; the source lock confirms redis==7.4.1, used by memory_persistent and graph_falkordb. That is the Python client version, not the Redis server version. [F]

Redis responsibility	What it means in code
Connection setup	Read host, credentials, TLS, and timeout settings. Prepare the client through the node lifecycle and close it when finished.
Tool input	Accept the agreed key, value, TTL, or other arguments. Validate them before calling Redis.
Storage call	Use redis-py methods such as get(), set(..., ex=...), delete(), and incr(), if those operations are approved.
Result or failure	Return a consistent result shape and an understandable error. Avoid exposing secrets in messages.
Access limits	Apply the configured key prefix and reject writes unless enabled. Bound scans, values, and deletion batches.

A proposed Redis read result

```
# Suggested contract, not an existing upstream Redis API
{"success": True, "found": True, "value": "saved answer"}
{"success": True, "found": False, "value": None}
```

The missing-key case is a normal lookup result. A failed connection is a failure. That distinction lets the app decide whether it needs to generate an answer, retry a safe read, or show an error. Final field names still need design sign-off.

Can Redis recognize paraphrases? A key lookup cannot do that automatically. Consistent keys or similarity matching are app logic. The brief leaves the required matching behavior ambiguous, so the team still needs a concrete example set. [H]

ROCKETRIDE EXPLAINED / 07

Setup and release: what is known?

Answer: the product, contributor, app, and release paths are documented; access and acceptance decisions still need owners.

Question	Answer already available
How do I try RocketRide?	Install the extension, open RocketRide in the sidebar, choose a Local runtime, and open a .pipe workflow. The quickstart describes the run controls. [G]
Do we need Redis Cloud to develop?	The client brief specifies local Redis in Docker, so no hosted Redis account is required for development. Docker is separate from the RocketRide engine. [H]
Must I compile C++?	The node is Python. The builder can use a prebuilt engine; a full source build has additional toolchain requirements. Use the contributor guide for that path. [D]
Where can a new node run before release?	A local engine can load local_nodes through --node_path. The documented module path is local_nodes.. Hosted apps cannot rely on that private local installation. [I]
How do we create the app?	Use App Builder's New App scaffold and pnpm. The guide covers preview, packaging, server builds, and publishing. [J]
How does the node reach others?	Contribute it to nodes/src/nodes// through a PR against develop, following upstream issue, branch, test, and README rules. RocketRide then releases it. [I]

Which tests should the Redis implementation cover?

The framework already separates manifest checks from live pipeline runs. Our proposed behavior tests should cover read/write/expiry, missing keys, prefix isolation, write-disabled mode, bad credentials, connection failures, and bounded operations. Counters need concurrency tests; avoid retrying an uncertain increment as though it were a harmless read. [D]

Release dependency already answered: the brief requires the hosted app to use the released catalog node. It explicitly provides App Builder preview as the demo fallback if release slips. That policy is known; the actual release owner and readiness still need confirmation. [H, J]

ROCKETRIDE EXPLAINED / 08

What should we still ask RocketRide?

Replace the broad example-node question with a few decisions that code and documentation cannot make for the team.

1. Confirm the first-version contract. Which operations and connection options are required? Should this first contribution follow the tool-node pattern demonstrated here, and do they also require ordinary data-lane behavior?

2. Settle the app and matching rules. Is Answer Cache selected? Which wording variations must share an answer, and which advanced features can wait? Is app-controlled lookup acceptable, or must an agent demonstrate it?

3. Agree on acceptance and access. What exact demo and test evidence count as complete? Who supplies development access, user/admin identity guidance, model access, and the hosted demo environment?

4. Name owners and confirm dates. Who approves the design, reviews the PR, and handles catalog release? Are the October 30, November 6, November 13, November 20, and November 25 targets still agreed?

A sharper replacement for the earlier question:
"We studied tool_notion and tested its request/result behavior offline. We plan to use the same Python tool pattern and direct client invocation for Redis. Is that the right interface for your intended workflows, and what additional acceptance checks do you want?"

Sources you can inspect

[A] Notion implementation; unit tests; contract checks

[B] Node architecture and tool connections

[C] Tools-source direct calls

[D] Test framework; contributor setup

[E] Python SDK use, validate, and tool methods

[F] Pinned dependency lock

[G] Quickstart: extension and local runtime

[I] Upstream contribution requirements

[J] App Builder guide

[H] Supplied RocketRide_AIC_Redis_Node_Integration_v2.pdf, especially pages 1-2 and 6-7. Test traces and results were produced locally on October 29, 2026.